

MINI-HACKATHON: Flowchart Master & C# Code Connection

Biến Ý Tưởng Thành Hiện Thực

Tiết 3 | 45 Phút

Mục tiêu: Chuyển hóa tư duy logic (CLO3) thành mã nguồn thực tế (CLO2).

Gemini Notebook

Nhiệm Vụ Và Luật Chơi (45 Phút)

Làm việc theo nhóm. Mục tiêu là giải quyết 3 bài toán thực tế bằng cách tuân thủ nghiêm ngặt quy trình rèn giữa logic trước khi gõ code.







Giai đoạn 1: Bản Vẽ (15 Phút)
Thiết kế sơ đồ khối (Flowchart) trên giấy/bảng.

Giai đoạn 2: Lò Luyện (10 Phút)
Đổi bài và phản biện chéo trong vai trò Trình biên dịch (Compiler).

Giai đoạn 3: Hiện Thực Hóa (20 Phút)
Chuyển giao sang mã giả và viết code C# trên Visual Studio.

Gemini Notebook

Ma Trận Thử Thách Logic

Mỗi bài toán kiểm tra một nhóm cấu trúc điều khiển nền tảng trong lập trình C#.



Bài toán 1: Đèn Giao Thông

Vấn đề: Nhập trạng thái đèn, in ra hành động tương ứng.

Trọng tâm Logic: Rẽ nhánh đa hướng.

Cấu trúc C#: Lệnh switch hoặc if-else if.



Bài toán 2: Tìm Số Lớn Nhất

Vấn đề: So sánh 3 số nguyên (a, b, c) để tìm ra số lớn nhất.

Trọng tâm Logic: Tuần tự & Logic so sánh.

Cấu trúc C#: Lệnh if lồng nhau & Toán tử &&.



Bài toán 3: Game Đoán Số

Vấn đề: Đoán số ngẫu nhiên (1-10) cho đến khi chính xác.

Trọng tâm Logic: Vòng lặp không xác định trước số lần.

Cấu trúc C#: Lệnh lặp while hoặc do-while.

Gemini Notebook

Bộ Công Cụ Dựng Sơ Đồ Khối



Bắt đầu / Kết thúc (Start/End)
Định tuyến điểm đầu và cuối của chương trình.

Nhập / Xuất (Input/Output)
Giao tiếp với người dùng (đọc dữ liệu, in kết quả).

Xử lý (Process)
Các phép gán, tính toán toán học.

Điều kiện (Decision)
Rẽ nhánh logic dựa trên kết quả True/False.

Gemini Notebook

Thử Thách 1: Hệ Thống Đèn Giao Thông



Yêu cầu bài toán:

Chương trình yêu cầu người dùng nhập vào màu của đèn giao thông hiện tại (**Đỏ, Vàng, Xanh**).

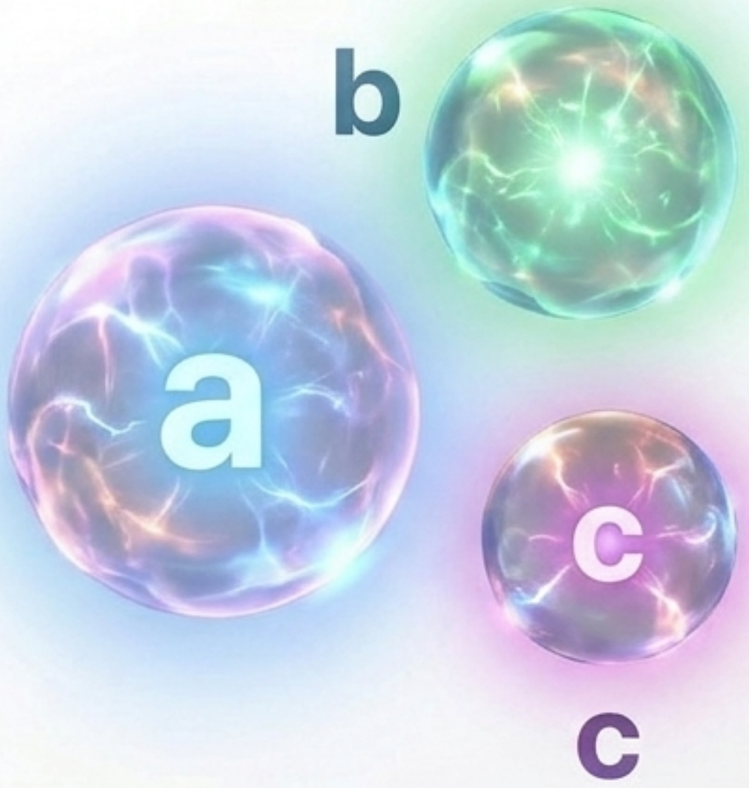
Dựa vào đầu vào, hệ thống phải in ra chỉ thị hành động tương ứng:

- **Đỏ**: Dừng lại
- **Vàng**: Đi chậm
- **Xanh**: Đi tiếp
- **Khác**: Trạng thái không hợp lệ

Gợi ý Logic: Từ một điểm nối, luồng xử lý của bạn sẽ chia thành bao nhiêu nhánh?

Gemini Notebook

Thử Thách 2: Truy Tìm Số Lớn Nhất



Yêu cầu bài toán:

Viết chương trình nhận vào 3 số nguyên ngẫu nhiên **a**, **b**, và **c**.

Xác định và in ra màn hình giá trị lớn nhất trong 3 số đó.

Gợi ý Logic & Bẫy (Edge Cases):

- Làm sao để sơ đồ khối thể hiện việc so sánh **a** với cả **b** và **c**?
- Hãy cẩn thận: Điều gì xảy ra nếu **a == b**, hoặc cả 3 số bằng nhau? Sơ đồ khối của bạn có bị kẹt (dead-end) không?

Gemini Notebook

Thử Thách 3: Game Đoán Số Ngẫu Nhiên



Yêu cầu bài toán:

Hệ thống (máy tính) sẽ tạo ra một số bí mật từ 1 đến 10.

Người dùng liên tục nhập một số để đoán. Hệ thống sẽ lặp lại yêu cầu nhập cho đến khi người dùng đoán chính xác.

Gợi ý Logic:

- Khác với rẽ nhánh, luồng (flow) của vòng lặp phải quay ngược lại một bước trước đó. Khối điều kiện của bạn sẽ nằm ở đầu vòng lặp hay cuối vòng lặp?

Gemini Notebook

Đổi Bài! Hãy Đóng Vai Trình Biên Dịch



Con người

Sáng tạo, linh hoạt, hay tự suy diễn ý định.



Trình Biên Dịch (Compiler)

Lạnh lùng, chính xác, tuyệt đối tuân thủ.

Quy tắc của Compiler: Trình biên dịch **không có trí tưởng tượng**. Trình biên dịch không tự hiểu ý của bạn. Trình biên dịch chỉ di chuyển **tuần tự theo đúng những mũi tên được vẽ trên giấy**. Nếu **không có mũi tên**, nó sẽ **Crash**.

Gemini Notebook

Kiểm Tra Lỗi Logic (Human vs. Compiler)

Sử dụng bảng sau để dò lỗi trên sơ đồ khối của nhóm bạn:

Góc nhìn Con Người (Human)	Góc nhìn Trình Biên Dịch (Compiler)	Lỗi Cần Bắt (Bugs)
Đương nhiên biến này là số.	Biến này đã được khởi tạo giá trị chưa?	 Lỗi cấp phát (Definite assignment)
Đường nào cũng ra kết quả.	Nếu Điều kiện là FALSE, nhánh này đi về đâu?	 Kẹt luồng (Dead-end path)
Lặp đến khi đoán đúng thì thôi.	Có biến nào thay đổi để điều kiện thoát lặp thành TRUE không?	 Vòng lặp vô hạn (Infinite Loop)

Gemini Notebook

Giải Mã: Từ Đồ Họa Đến Mã Nguồn

Mã giả (Pseudocode) là bước đệm an toàn giữa tư duy không gian và cú pháp C#.

1. Ý Tưởng (Flowchart)



2. Mã Giả (Pseudocode)

```
NẾU x lớn hơn 5 THÌ
    Thực hiện khối lệnh
```

3. Mã C# (Syntax)

```
if (x > 5)
{
    // Thực hiện khối lệnh
}
```

Gemini Notebook

Hộp Đồ Nghề Cú Pháp C# (Sẵn Sàng Code)

Lắp ráp các cấu trúc sau vào Visual Studio để giải quyết 3 bài toán:

Nhập / Xuất

```
Console.ReadLine();
Console.WriteLine();
```

Chuyển kiểu (Parsing)

```
int.Parse(...)
```

Rẽ nhánh & Lựa chọn

```
if (...) { }
} else { }

switch (biến) {
    case x: break;
}
```

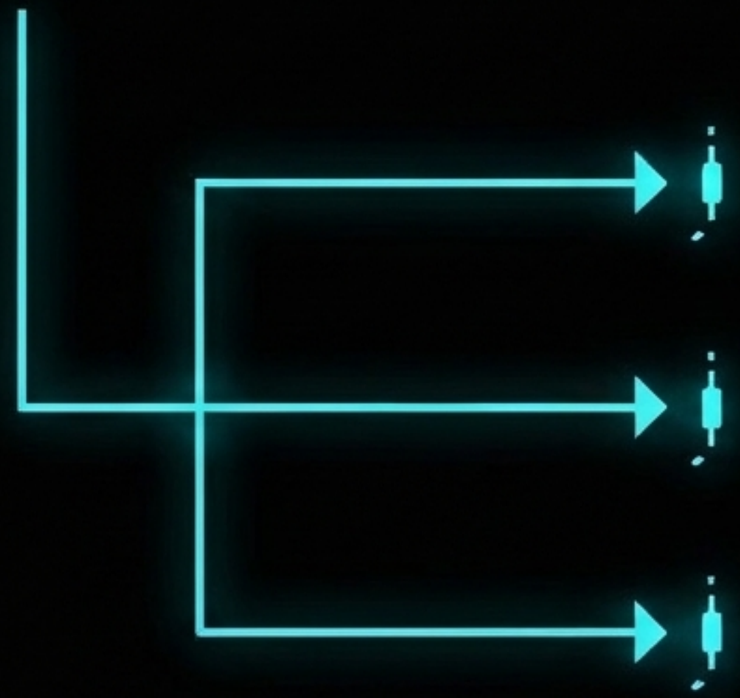
Vòng lặp

```
while (điều kiện) { }
do { }
} while (điều kiện);
```

Gemini Notebook

Giải Pháp 1: Lựa Chọn Đa Hướng (Switch)

Thay vì dùng nhiều lệnh if-else lồng nhau gây rối mắt, switch kiểm tra một giá trị rời rạc với độ đọc hiểu cao hơn.



```
string color = Console.ReadLine();
switch (color)
{
    case "Đỏ":
        Console.WriteLine("Dừng lại");
        break;
    case "Vàng":
        Console.WriteLine("Đi chậm");
        break;
    case "Xanh":
        Console.WriteLine("Đi tiếp");
        break;
    default:
        Console.WriteLine("Lỗi");
        break;
}
```

Gemini Notebook

Giải Pháp 2: Toán Tử Logic (&&)

Hai khối điều kiện lồng nhau trong sơ đồ khối có thể được gộp lại thành một biểu thức duy nhất bằng toán tử AND (&&).




```
int a = int.Parse(Console.ReadLine());
int b = int.Parse(Console.ReadLine());
int c = int.Parse(Console.ReadLine());

if (a >= b && a >= c) {
    Console.WriteLine($"Lớn nhất là: {a}");
} else if (b >= a && b >= c) {
    Console.WriteLine($"Lớn nhất là: {b}");
} else {
    Console.WriteLine($"Lớn nhất là: {c}");
}
```

Gemini Notebook

Giải Pháp 3: Vòng Lặp Post-Test (do-while)

Vì sao dùng do-while thay vì while? Trò chơi yêu cầu người dùng phải nhập số đoán ít nhất một lần trước khi kiểm tra điều kiện thắng.



```
Random rnd = new Random();
int secret = rnd.Next(1, 11);
int guess;

do
{
    Console.Write("Đoán số (1-10): ");
    guess = int.Parse(Console.ReadLine());
} while (guess != secret);

Console.WriteLine("Chính xác!");
```

Gemini Notebook

Tổng Kết: Quy Trình Kỹ Thuật Lập Trình



1. **Logic đi trước Cú pháp**: Vẽ sơ đồ khối để triệt tiêu lỗi tư duy trước khi chạm vào bàn phím.
2. **Thiết kế phòng thủ:** Trình biên dịch hiểu nghĩa đen. Hãy khởi tạo biến và kiểm soát điều kiện thoát vòng lặp.
3. **Lắp ghép nền tảng:** If/Else, Switch, và Vòng lặp chính là những viên gạch xây dựng nên mọi thuật toán phức tạp nhất.

Nhiệm vụ tuần tới: Ứng dụng các vòng lặp này để duyệt và xử lý dữ liệu với Mảng (Arrays)!

Gemini Notebook